

Action-Prefix System — User Guide

Field	Value
Revision	1
Created	2026-06-09
Last modified	2026-06-09T00:00:00Z
Status	active
Scope	end-user guide to the universal ACTION_NAME :: prompt-prefix system (§11.4.140)
Audience	end users of any CLI agent that includes the constitution submodule
Inputs	docs/research/action_prefix_system/ {DESIGN.md, GRAMMAR_ADDENDUM.md, IMPLEMENTATION_REPORT.md}

Anti-bluff (§11.4.6): this guide documents only what the design + the verified implementation actually do. Where a feature is designed but not yet implemented, that is stated explicitly as **(spec-only)** — never implied to work. The honest §11.4.3 per-agent support boundary is in §6.

1. What the action-prefix system is

The action-prefix system lets you put a short, UPPERCASE **action keyword** at the very start of your prompt to apply a registered behaviour to the rest of it — without retyping a long instruction every time.

When your prompt's first non-blank line starts with an action keyword followed by :: (one space, two colons, one space), the agent looks the keyword up in a shared **action registry**, replaces the ACTION_NAME :: prefix with that action's full **expansion text**, and then executes the rest of your prompt under that expanded instruction.

Example. You type:

BACKGROUND :: Investigate the audio-routing regression on D3 and propose a fix.

The agent expands BACKGROUND :: into the registered BACKGROUND instruction (run as a background, subagent-driven, parallel work stream that produces rock-solid physical proof — see §3) and then performs your actual task (“*Investigate the audio-routing regression on D3 and propose a fix*”) under that instruction.

The system is **universal** (works across every supported CLI agent), **extensible** (new actions are added by editing one data file, never code — see the [Manual](#)), and **loads out-of-the-box** once the constitution submodule is included in a project (see §7).

Canonical authority: constitution submodule §11.4.140 (Constitution.md).

2. The four equivalent invocation forms

Every registered action is **designed** to be invocable in four equivalent forms — same action, same expansion, same execution. The first non-blank line of your prompt is what is matched.

#	Form	Example	Notes	Status
1	ACTION_NAME :: <rest>	BACKGROUND :: Do X	bare :: form (no namespace)	Implemented
2	PREFIX::ACTION_NAME :: <rest>	DEFAULT::BACKGROUND :: Do X	namespaced :: form	Implemented (grammar 91/0; see §6)
3	/ACTION_NAME <rest>	/BACKGROUND Do X	bare slash form — ONLY if /ACTION_NAME does NOT collide with an existing/ built-in slash command of the host agent	Implemented (generated slash command; §6)
4	/PREFIX::ACTION_NAME <rest>	/DEFAULT::BACKGROUND Do X	namespaced slash form — disambiguates from any existing / ACTION_NAME; ALWAYS safe	Implemented (hook E2E 10/0; see §6)

These four forms are equivalent by design:

```
BACKGROUND :: ≡ DEFAULT::BACKGROUND :: ≡ /BACKGROUND ≡ /  
DEFAULT::BACKGROUND
```

Honest status note (§11.4.6). All four forms are implemented today and work end-to-end. Forms **2 and 4** (the `DEFAULT::` namespace) are now wired into the registry, the expander library, and the slash-command generator, and are verified (grammar suite 91/0, hook E2E 10/0) — see §8. Any of the four forms can be used today.

2.1 The separators (do not confuse them)

- **Namespace separator** — `::` with **no surrounding spaces**, used **INSIDE** the token: `PREFIX::ACTION_NAME` (forms 2 and 4).
- **Action-body separator** — `::` with **a space on each side**, used between the token and the rest of your task in the `::` forms (forms 1 and 2).
- **Slash-form body separator** — a single space, between the slashed token and the rest (forms 3 and 4): `/BACKGROUND Do X`.

The exact `::` (space-colon-colon-space) separator in forms 1 and 2 is chosen deliberately so the prefix never collides with C++ `Foo::Bar`, YAML key: `value`, URLs, or 12::34-style typos.

3. The BACKGROUND action's effect

`BACKGROUND` is the first registered action. Its expansion text (verbatim from the registry) is:

*“The following prompt that we will provide **MUST BE** executed in background in parallel with all main work streams using the subagents-driven development approach! All work done **MUST PRODUCE** rock solid evidence covered with hard physical proof(s) that all done is working as expected and as specified without any false results and without any bluff!”*

In plain terms, prefixing your task with `BACKGROUND ::` tells the agent to:

- run the task as a **background work stream** in parallel with other work,
- drive it via the **subagent-driven** approach,
- and produce **rock-solid physical proof** that the result really works — no false results, no bluff.

It composes with the constitution's subagent-driven (§11.4.20 / §11.4.70), parallel-stream (§11.4.58 / §11.4.103), background-execution (§11.4.89), and captured-evidence (§11.4.5 / §11.4.69 / §11.4.107) + anti-bluff (§11.4) anchors.

More actions can be added later; each gets its own keyword, expansion text, and optional rules. See the Manual §“How to add a new action”.

4. The escape — discussing an action without triggering it

If you want to *write about* an action keyword without triggering its expansion (for example, to ask a question about it), put a single backslash `\` at the very start of the line:

```
\BACKGROUND :: x
```

The agent strips the leading backslash and treats the line as the ordinary prompt `BACKGROUND :: x` — **no expansion happens**. The same escape applies to the slash forms: `\/BACKGROUND` `x` is literal.

5. Other grammar rules you should know

These rules keep the system predictable and prevent accidental triggers:

1. **Anchored at the first non-blank line only.** The keyword must be the very start of your prompt's first non-blank line. `please BACKGROUND :: x` does **not** trigger — the keyword is not at line start, so it is an ordinary prompt. Leading blank lines before the keyword are allowed.
2. **UPPERCASE only.** Action keywords are `[A-Z][A-Z0-9_]*`. `background :: x` (lowercase) does **not** trigger — this makes the keyword visually unmistakable and avoids matching ordinary prose.
3. **Exact separator.** The `::` forms require exactly `::` (one space each side). `BACKGROUND::do X` (no spaces) does **not** trigger.
4. **Stacked keywords.** `OUTER :: INNER :: do X` applies outer-to-inner, left-to-right: expand `OUTER`, re-scan the remainder, expand `INNER`, then `do X` is the task. Both expansions apply. (*In practice only BACKGROUND is registered today, so stacking is available but rarely used until more actions land.*)
5. **Unknown keyword — never guessed.** If your first-line token looks like an action (`SOMETHING :: ...`) but is **not** in the registry,

the agent will **not** silently expand it or silently drop it. It asks which registered action you meant (per §11.4.66 / §11.4.105), or treats your line literally — it never invents an expansion (§11.4.6). So a typo like `BACKGROUND :: x` gets a clarifying question naming the closest registered action, not a guess.

6. **No-op when nothing matches.** Any prompt that does not satisfy the grammar is just an ordinary prompt; the system does nothing.

6. Per-agent support matrix

The system is built in two layers (full architecture in the [Manual](#)):

- **LAYER 1 (always-on, every agent):** a recognition instruction is embedded in every agent context carrier (`CLAUDE.md`, `AGENTS.md`, `QWEN.md`, `GEMINI.md`). The agent reads it on every prompt and applies the free-form prefix **itself**. This is the universal floor — it works on every agent with zero extra setup.
- **LAYER 2 (mechanical, where the agent exposes a pre-submit seam):** on Claude Code a `UserPromptSubmit` hook rewrites/injects the expansion deterministically before the model sees the prompt; on Gemini/Qwen/Codex a generated **slash command** (`/background`) is the mechanical convenience.

Agent	Free-form <code>BACKGROUND :: ...</code> (form 1)	Mechanical interception	Slash command (form 3)
Claude Code	LAYER 1 + LAYER 2 hook (transparent)	YES — <code>UserPromptSubmit</code> hook rewrites before the model sees it	(hook covers it)
Gemini CLI	LAYER 1 (agent self- applies)	NO transparent rewrite	<code>/background</code> (generated <code>.gemini/ commands/ background.toml</code>)
Qwen Code	LAYER 1 (agent self- applies)	NO transparent rewrite	<code>/background</code> (generated <code>.qwen/ commands/ background.toml</code>)
OpenAI Codex CLI	LAYER 1 via <code>AGENTS.md</code>	NO transparent rewrite	<code>/</code> <code>prompts:background</code> (generated <code>prompts/ background.md</code>)
	LAYER 1 via <code>AGENTS.md</code> +	NO	

Agent	Free-form BACKGROUND :: ... (form 1)	Mechanical interception	Slash command (form 3)
GitHub Copilot CLI	copilot- instructions.md		(custom agent optional)
Cursor	LAYER 1 via .cursor/rules + AGENTS.md	NO	—
Aider	LAYER 1 via CONVENTIONS.md / AGENTS.md	NO	—
Cline / Continue / Roo	LAYER 1 via .clinerules / .continue/ rules / AGENTS.md	NO	—

6.1 Honest §11.4.3 boundary — Claude-Code-only mechanical rewrite

Transparent, mechanical free-form interception — the BACKGROUND :: ... string being rewritten *before the model ever sees it* — is genuinely available **ONLY on Claude Code**, because Claude Code is the only agent (verified against its official hooks documentation) that exposes a pre-submit UserPromptSubmit seam.

This is **not a gap**:

- On every other agent the free-form form still works — LAYER 1 makes the agent recognise and apply the prefix itself (it reads the recognition instruction from its context carrier).
- The generated **slash command** (/background, form 3) is the mechanical convenience on the agents that support generated commands (Gemini, Qwen, Codex).

So: **form 1 (free-form) works everywhere via LAYER 1; mechanical rewrite is a Claude-Code upgrade; the slash command is the convenience elsewhere.** The slash generators are a nicety, not the load-bearing path.

7. Quick-start — how it loads out-of-the-box

You do not install anything per-prompt. Once a project includes the constitution submodule, the system is available:

1. **LAYER 1 is automatic.** The recognition instruction lives in the project's `CLAUDE.md` / `AGENTS.md` / `QWEN.md` / `GEMINI.md` (propagated per §11.4.35). Every agent loads its carrier with every prompt, so it already knows the prefix grammar and the BACKGROUND expansion — even without registry-file access (the BACKGROUND expansion is inlined into the carrier block).
2. **LAYER 2 is wired by the installer.** Running the project setup invokes `scripts/install_action_prefix.sh` (by reference, per §11.4.80), which:
 - merges the Claude Code `UserPromptSubmit` hook entry into the project's `.claude/settings.json` (idempotent — no duplicate on re-run, pre-existing hooks preserved, `settings.json` backed up first), and
 - runs the slash-command generator so `/background` exists for the other agents.
3. **Use it.** Start a prompt with `BACKGROUND :: <your task>` (form 1) on any agent, or `/background <your task>` (form 3) on Gemini/Qwen/Codex.

To verify the system is live, see the [Manual](#) §“Troubleshooting”.

8. What is implemented (read before relying on a form)

Per §11.4.6 this is stated as fact, not implied:

Implemented + verified working (per the Implementation Report):

- Form 1 — `ACTION_NAME :: <rest>` — end-to-end: registry → expander library → LAYER-1 recognition instruction → LAYER-2 Claude Code hook.
- Form 2 — `PREFIX::ACTION_NAME :: <rest>` (e.g. `DEFAULT::BACKGROUND :: ...`) — namespaced :: form (grammar suite 91/0).
- Form 3 — `/ACTION_NAME <rest>` — the generated bare slash command (`/background`) for Gemini (`.toml`), Qwen (`.toml`), Codex (prompts/).
- Form 4 — `/PREFIX::ACTION_NAME <rest>` (e.g. `/DEFAULT::BACKGROUND ...`) — namespaced slash form, always-unambiguous (hook E2E 10/0).

- The DEFAULT namespace and the registry keys that drive forms 2/4 (grammar.default_namespace, grammar.namespace_separator, grammar.slash_prefix, per-action namespaces, slash_bare, slash_conflicts), the conflict-aware bare-slash rule (slash_bare: auto), and the /default::background namespaced slash artefact.
- The BACKGROUND action with the operator's verbatim expansion.
- The escape (\), stacked-prefix (outer-to-inner), unknown-token-ask, and no-op rules of §5.

All four forms are verified end-to-end (grammar suite 91/0, hook E2E 10/0). The registry declares the full namespaced grammar and the expander library's `apx_parse_prefix` recognises all four forms; there is no remaining spec-only gap in the grammar.

9. Related documents

- [MANUAL.md](#) — developer/maintainer manual (architecture, registry schema, adding a new action, conflict rule, troubleshooting).
- [DIAGRAMS.md](#) — Mermaid diagrams of the expansion flow, the two-layer architecture, the 4-form grammar decision tree, and the add-a-new-action sequence.
- Source design: docs/research/action_prefix_system/{RESEARCH.md, DESIGN.md, GRAMMAR_ADDENDUM.md, IMPLEMENTATION_REPORT.md, RULE_DRAFT.md}.
- Canonical authority: Constitution.md §11.4.140.